武漢大學
WUHAN UNIVERSITY

《分布式软件原理与技术》
第四讲 高并发设计（3）

计算机学院 贾向阳

武漢大學
WUHAN UNIVERSITY

1、 异步与消息队列

异步写

异步写

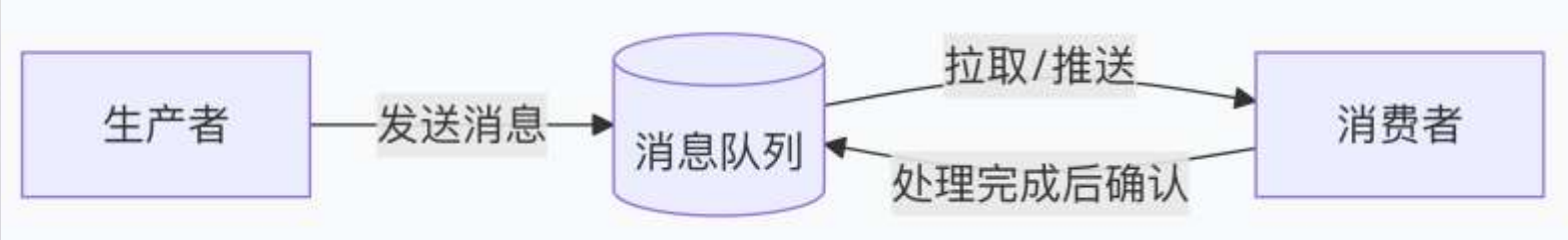
- **“取号离场”模式**：把需求提交给柜台（消息队列），对方返回回执说“收到了”，你就可以先去干别的事。等后台处理完了，再通知你或你主动查询。

问题	同步写痛点	异步写解决方案
响应时间	需等待所有下游完成，RT可达数秒	写入队列即返回，RT降至毫秒级
系统耦合	上游强依赖下游，一个故障全链路阻塞	解耦生产者与消费者，各自独立演进
流量冲击	突发流量直接打垮数据库	队列缓冲，消费者按自身能力拉取
资源利用率	线程阻塞等待，浪费系统资源	异步非阻塞，线程可处理其他任务

消息队列

消息队列（Message Queue，MQ）

- 是一种跨进程通信的中间件，用于在分布式系统中异步传递消息。
- 采用先进先出（FIFO）的数据结构，将消息存储在队列中，生产者将消息放入队列，消费者从队列中取出消息进行处理。



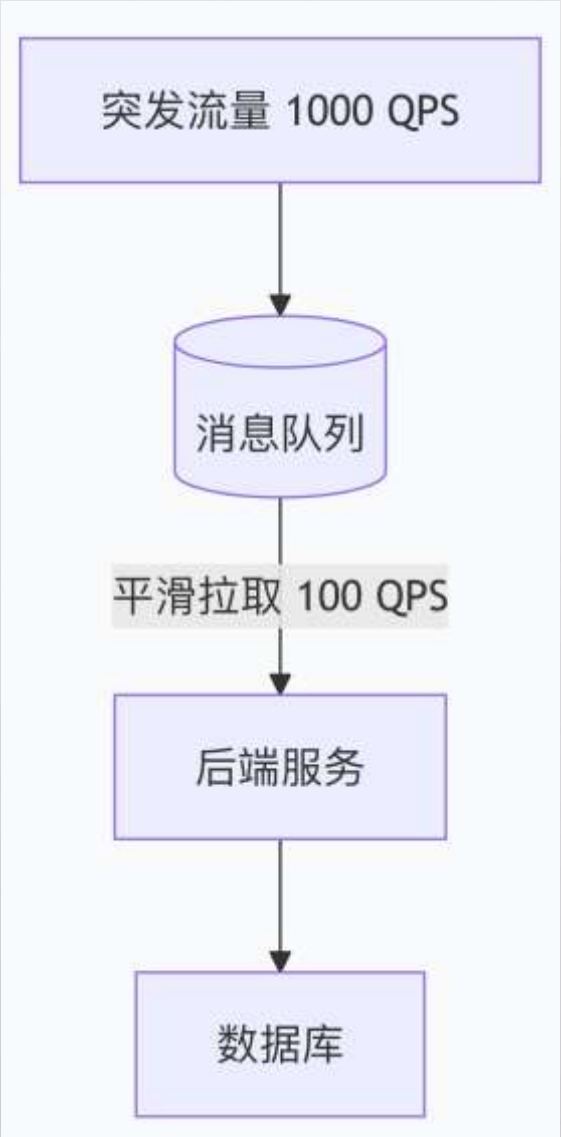
两种消息模型

- 发布订阅模型（Topic）：一条消息可被多个消费者消费，消息广播给所有消费者。
- 点对点模型（Queue）：一条消息只被一个消费者消费，消费后消息从队列移除。不同的消息中间件实现方式不同。

武漢大學 WUHAN UNIVERSITY

消息队列的作用

1. **解耦**
   - 生产者和消费者不直接依赖，通过队列隔离。
   - 修改消费者逻辑不影响生产者，反之亦然。
   - 降低系统间的耦合度，提升可维护性。
2. **异步**
   - 生产者发送消息后立即返回，无需等待消费者处理。
   - 缩短主流程响应时间，提升用户体验。
3. **最终一致性**
   - 支持分布式事务的最终一致性（如订单创建后异步发送短信、积分）
   - 通过消息重试和死信队列保障消息可靠处理。
4. **削峰填谷**
   - 把高峰并发请求在消息队列中排队，请求被快速响应。
   - 后端以稳定的速度处理请求，在低谷期把堆积的工作做完。
   - 通过拉长时间维度，平衡了资源的利用率



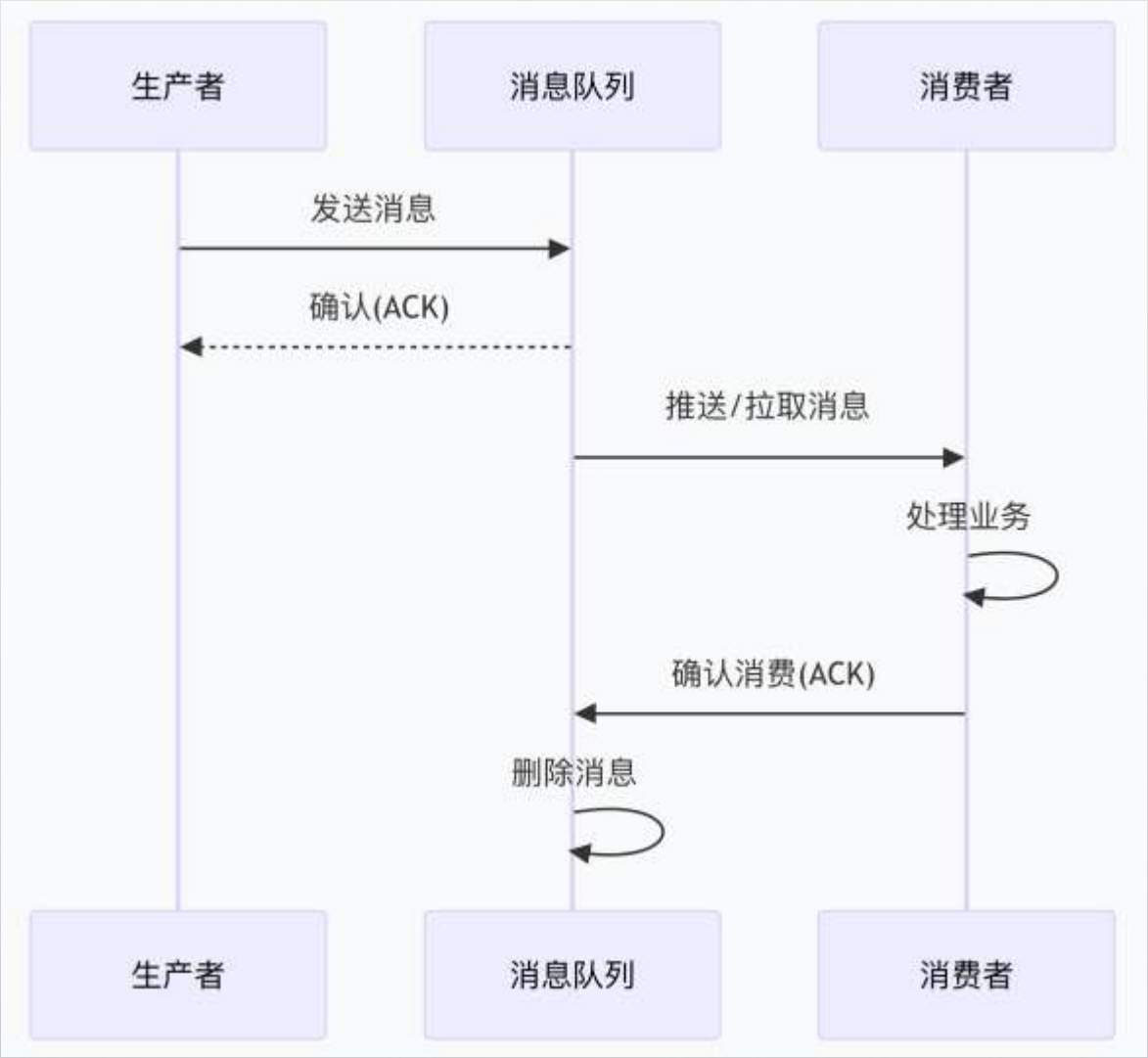
消息队列的工作原理

1. 消息存储

- 消息通常持久化到磁盘，防止宕机丢失。
- 采用顺序写（Append-Only）的方式写入文件，获得极高写入性能
- 内存中维护索引或缓存，加速读取

2. 消息确认机制（ACK）

- 生产确认：生产者收到队列的ACK后才认为消息发送成功。
- 消费确认：消费者处理完成后向队列发送ACK，队列才删除消息，防止处理过程中消费者宕机导致消息丢失。



消息队列的工作原理

3. 消息可靠性保证

- 持久化：消息写入磁盘
- 副本机制：多副本同步，确保某节点宕机消息不丢。
- 重试与死信：消费失败自动重试，多次失败进入死信队列人工处理。

4. 消息顺序性

- Kafka：分区（相当于银行的多个窗口）内有序。生产者写入时追加到分区末尾，消费者按偏移量顺序拉取。
- RabbitMQ：单一队列内保证顺序（FIFO）
- RocketMQ 提供了两种顺序保障
  - 分区顺序：与 Kafka 类似
  - 全局顺序：将一个主题的队列数设为 1，所有消息进入同一队列，生产者单线程发送，消费者单线程消费，牺牲吞吐量换取全局顺序

消息队列技术选型

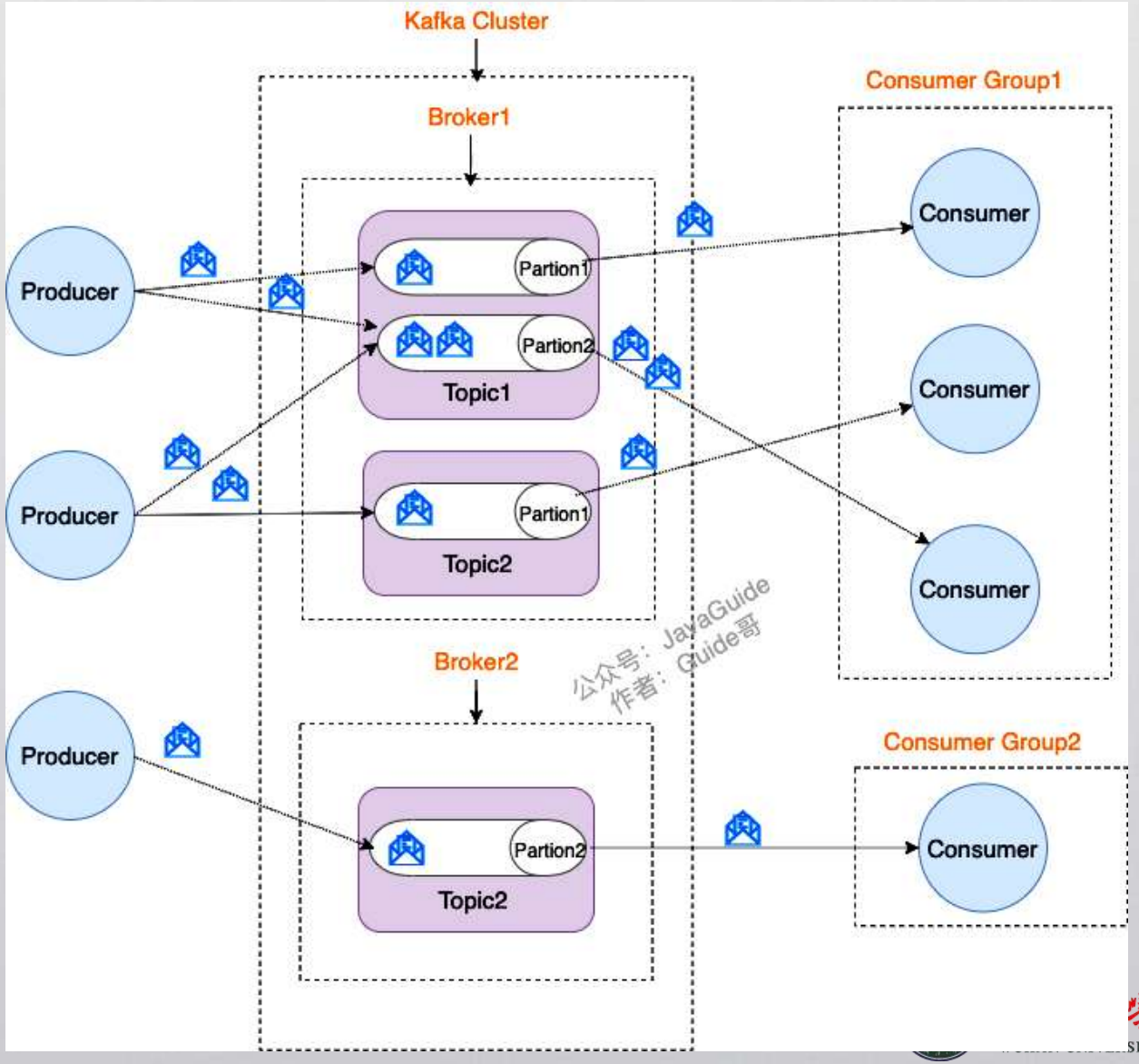
对比维度	Kafka	RabbitMQ	RocketMQ
设计定位	高吞吐流处理平台	通用消息代理	金融级分布式消息
吞吐量(TPS)	100万+（最高）	5-10万	50万+
延迟	<10ms	50-100ms	20-50ms
消息顺序	分区内有序	队列内有序	支持严格全局有序
事务消息	需外部组件	不支持	原生支持
定时/延迟消息	需外部组件	插件支持	原生支持
协议支持	Kafka协议	AMQP/MQTT/STOMP	自定义TCP
运维复杂度	高（需管理分区）	中	中高
优势	百万级吞吐，与Flink/Spark生态无缝对接	灵活路由、易用性强，支持MQTT协议	事务+顺序+高可靠，满足金融级要求
场景	日志收集/大数据流处理	业务系统解耦/任务队列，物联网设备接入	电商交易/订单系统支付与金融核心

注：Redis提供了LIST和Stream，可以实现消息队列功能，但可靠性要自己处理，慎用于重要业务。

Kafka中工作机制

基本概念

- **Broker**：Kafka服务器实例
- **Producer**：消息生产者
- **Consumer**：消息消费者
- **Topic**：主题。生产者可以通过主题推送给多个消费者
- **Partition**：主题的分区。相当于银行的多个窗口，每个Partition维护一个队列。
- **Replicas**: 主题的副本数，支持高可用。
- **Consumer Group**：消费者的逻辑分组，实现消费者负载均衡和容错。 topic的一个消息由组内的任一个消费者处理。



Kafka的工作机制

高效读写与持久化机制

- **消息存储：**每个分区对应一个目录，目录下包含多个日志段（Segment），消息顺序追加到当前活跃的 Segment，写入完成后不可修改
- **顺序写：**采用顺序写（Append-Only）的方式写入文件，获得极高写入性能
- **页缓存：**内存中维护索引与缓存（Page Cache），加速读取
- **零拷贝：**消费数据时，数据直接从页缓存发送到网络 socket，避免数据在内核态和用户态之间拷贝，降低 CPU 开销

Kafka的工作机制

- **分区机制**
  - **并行度：**主题的分区数决定了最大消费并行度（一个分区只能被一个消费者消费）
  - **负载均衡：**生产者通过 key 将消息路由到特定分区，实现业务级别的有序性
- **副本机制**
  - 每个分区有多个副本（replica），其中一个是 leader，其余是 follower
  - 所有读写操作都通过 leader 进行，follower 异步拉取数据保持同步。当 leader 宕机时，从 follower 中选举新 leader

Kafka的工作机制

- **消费者组——实现负载均衡与容错**
  - 每个消费者属于一个组，组内消费者共同消费一个或多个主题
  - 分区分配：一个分区只能分配给组内一个消费者
  - 负载均衡：当组内消费者数量变化时，触发再均衡（rebalance），重新分配分区
- **消费进度管理**
  - 消费者提交当前消费的 offset 到 Kafka 内部主题 __consumer_offsets
  - 重启后从上次提交的 offset 继续消费
  - 可手动提交（精确控制）或自动提交（简单但可能丢或重复）
- **消费语义**
  - 至少一次（at-least-once）：可能重复，需要幂等
  - 至多一次（at-most-once）：可能丢，适合不敏感的日志
  - 精确一次（exactly-once）：需要事务和幂等，Kafka 0.11+ 支持

SpringBoot中使用Kafka消息队列

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092   # Kafka 服务器地址
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: 1                           # 可靠性：只要leader确认即可
      retries: 3                         # 发送失败重试次数
    consumer:
      group-id: seckill-group            # 消费者组ID
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "*"  # 信任所有包（反序列化用，可以设置某些包防止反序列化漏洞）
      enable-auto-commit: false          # 关闭自动提交消费进度（偏移量），设为手动ACK
      auto-offset-reset: latest          # 无偏移量时从最新开始消费，忽略已有堆积
    listener:
      type: batch                        # 批量消费，一次拉取多个消息
      concurrency: 3                      # 并发消费者数（不能超过主题的分区总数）
```

武漢大學
HAN UNIVERSITY

SpringBoot中使用Kafka消息队列

```java
@Configuration
public class KafkaConfig {

    @Bean
    public NewTopic seckillTopic() {
        return TopicBuilder.name("seckill-orders")
                .partitions(6)              // 设置6个分区，支持并行消费
                .replicas(1)                // 副本数，生产环境建议2-3
                .build();
    }
}
```

```java
@Service
public class OrderProducer {

    @Autowired
    private KafkaTemplate<String, Object> kafkaTemplate;

    public void sendOrder(String userId) {
        // 构造消息对象
        OrderMessage message = new OrderMessage(userId, System.currentTimeMillis());
        // 发送到指定主题
        kafkaTemplate.send("seckill-orders", userId, message);  // 用userId作为key保证同一用户有序
        System.out.println("订单消息已发送: " + userId);
    }
}
```

SpringBoot中使用Kafka消息队列

```java
@Autowired
private OrderProducer orderProducer;

@GetMapping("/order")
public String seckillOrder(@RequestParam S
    // 1. 简单前置校验（如限流、黑名单）

    // 2. 将请求发往Kafka，立即返回
    orderProducer.sendOrder(userId);

    // 3. 告知用户排队中
    return "订单处理中，请稍后查看结果";
}
```

```java
@Service
public class OrderConsumer {
    private static final Logger logger = LoggerFactory.getLogger(OrderConsumer.class);
    @Autowired
    private OrderService orderService;

    @KafkaListener(topics = "seckill-orders", groupId = "seckill-group")
    public void processOrder(List<OrderMessage> messages, Acknowledgment ack) {
        logger.info("收到批量消息，数量：{}", messages.size());
        for (OrderMessage msg : messages) {
            try {
                String userId = msg.getUserId();
                // 幂等性检查（用Redis或数据库去重）
                if (orderService.isProcessed(userId)) {
                    logger.info("用户已处理，跳过：{}", userId);
                    continue;
                }
                orderService.createOrder(userId);
            } catch (Exception e) {
                logger.error("处理用户失败，userId={}", msg.getUserId(), e);
            }
        }
        ack.acknowledge();
    }
}
```

消息队列的问题处理

（1）消息重试造成乱序

问题：消息消费失败，重试机制将其重新入队，可能被再次消费，而后续消息已先被处理，导致乱序

解决：失败消息发送到重试队列或死信队列，暂停当前队列消费，直到成功（可能阻塞后续消息）。或者在数据库中通过版本号/状态约束来实现（高并发情况下推荐）。

示例：用户 A 下单（Msg1）后立刻取消（Msg2）。若 Msg1 消费失败触发重试并回到队列尾部，Msg2 可能先执行，导致系统先取消了一个不存在的订单，随后又创建了订单

```java
// 处理 Msg2: 取消消息
@Transactional
public void handleCancelOrder(OrderEvent event) {
    // 关键 SQL: update t_order set status = 'CANCEL' where order_id = ?
    // AND status IN ('INIT', 'PAID')
    // 如果状态已经是 CANCEL 或者记录不存在，则不更新
    int rows = orderMapper.updateStatus(event.getOrderId(), "CANCEL", Arrays.asList("INIT", "PAID"));

    if (rows == 0) {
        // 说明此时 Msg1 还没到（库里没记录），或者已经取消了
        log.info("取消消息先于下单消息到达，尝试创建“占位”取消记录");
        orderMapper.insertIgnore(new Order(event.getOrderId(), "CANCEL"));
    }
}
```

消息队列的问题处理

（2）消费者并发造成乱序

- 问题：同一队列有多个消费者并行处理，无法保证顺序。
- 场景： 同一个秒杀 Topic 有 2 个消费者线程。Msg1（下单）先发给 Thread-A，Msg2（支付）后发给 Thread-B。若 A 阻塞，B 先完成，状态机就会乱掉
- 解决：利用 Kafka 的 Partition 机制。确保同一个 orderId 的消息永远发送到同一个分区，且一个分区只由一个消费者处理。

```java
public void sendOrderMessage(OrderEvent event) {
    // 使用 orderId 作为 Key，Kafka 保证相同 Key 进入相同 Partition
    kafkaTemplate.send("order-topic", event.getOrderId(), JSON.toJSONString(event));
}
```

```java
@KafkaListener(
    topics = "seckill-topic",
    groupId = "seckill-group",
    concurrency = "3" // 假设有 3 个分区，这里启动 3 个线程并行执行
)
public void listen(ConsumerRecord<String, String> record) {
    // 因为同一个 orderId 都在同一个分区，
    // 这个线程会按顺序处理该 orderId 的所有消息
    processBusiness(record.value());
}
```

注意：在消费者方法内部不要开多线程，否则 Kafka 提供的“分区顺序保证”就失效了

消息队列的问题处理

（3）网络重传造成消息重复与乱序

- 问题： 生产者发送时因超时重发，可能导致消息重复且顺序交叉。
- 场景： 生产者发送“扣减库存”指令，因网络抖动未收到 ACK 从而触发重发。Kakfa此时可能存了两条指令，或者第二条先到达。
- 解决：幂等设计 + 序列号比对，在消费端去重并拒绝过期消息。

```java
@Transactional
public void processSeckill(OrderEvent event) {
    // 1. 幂等性检查：判断该序列号是否已处理
    if (redisService.hasProcessed(event.getSequenceId())) {
        return;
    }
    // 2. 序列号比对：只处理版本号 > 当前库中版本的数据
    int currentVersion = db.getOrderVersion(event.getOrderId());
    if (event.getVersion() <= currentVersion) {
        log.warn("拒绝过期消息: {}", event.getVersion());
        return;
    }
    // 执行业务并记录序列号
    updateDatabase(event);
    redisService.markAsProcessed(event.getSequenceId());
}
```

消息队列的问题处理

(4) 消息积压与跳过

- 如果某条消息长时间处理失败，后续消息可能被跳过或积压

- 场景： 秒杀峰值时，某条消息因为数据库死锁一直处理失败。如果简单地“跳过”去处理下一条，会导致数据不一致；如果一直卡着，后续几万条秒杀请求会堆积。
- 解决思路： 非阻塞式消息重试机制。若超过最大重试次数仍失败，转入死信队列人工干预，而非直接丢弃。

```java
@RetryableTopic(
    attempts = "3",
    backoff = @Backoff(delay = 1000, multiplier = 2),//退避策略（首次重试时间间隔1秒）
    topicStrategy = TopicStrategy.SINGLE_TOPIC,//创建一个统一的重试Topic
    dltStrategy = DltStrategy.FAIL_ON_ERROR // 最终失败进入 DLT
)
@KafkaListener(topics = "seckill-check")
public void handle(String message) {
    // 如果处理逻辑一直失败，Spring Kafka 会自动在 3 次尝试后将消息移出主流
    // 进入 "seckill-check-dlt" 主题，避免阻塞后续正常的秒杀消息
    verifyInventory(message);
}

@DltHandler
public void processDlt(String message) {
    // 记录日志，发送告警，等待人工修复数据
    log.error("秒杀核心消息彻底失败，进入死信队列：{}", message);
}
```

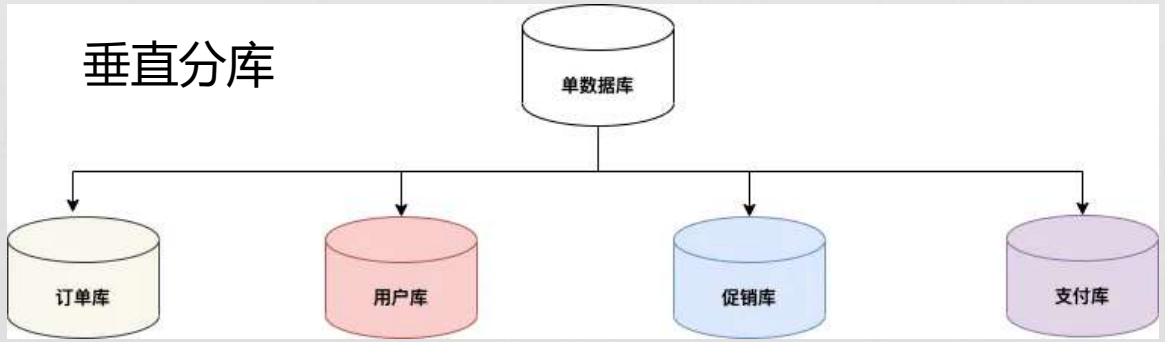
2、 分库分表

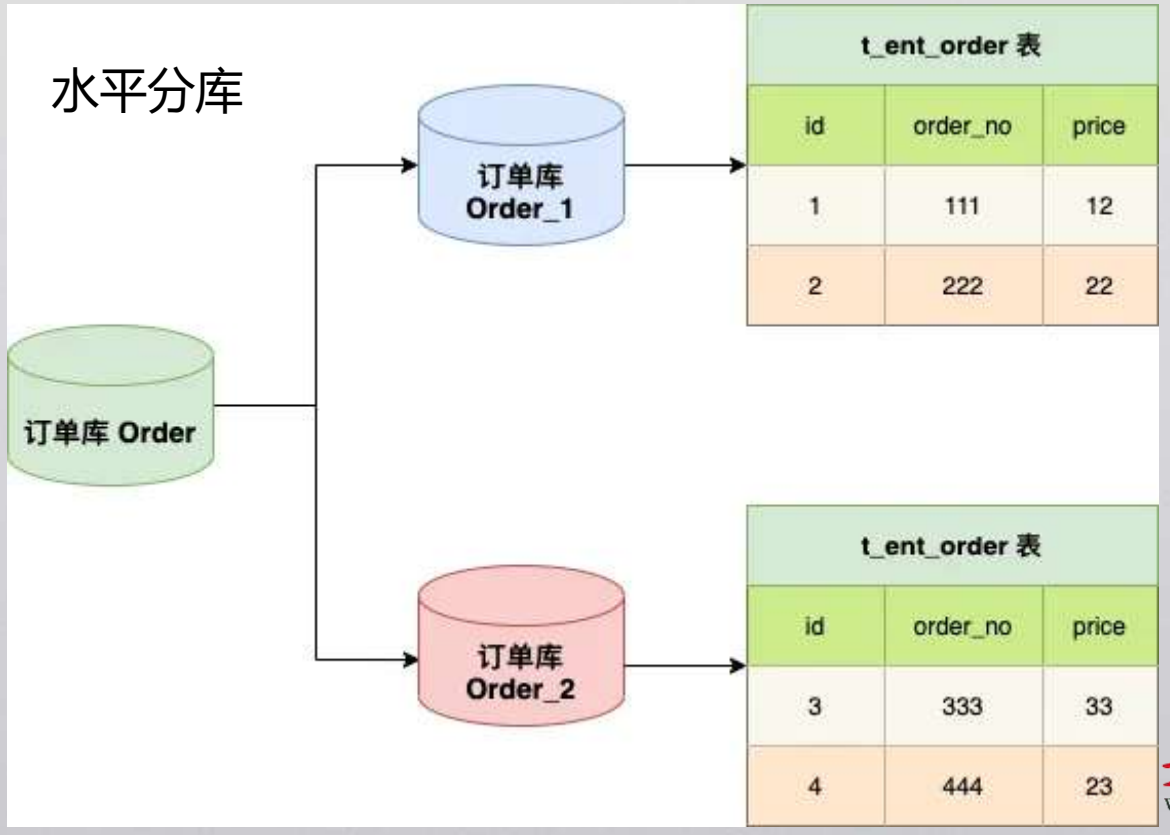
什么是分库分表?

- **为什么要分库分表?**
  - 单表数据量过大（如超过5000万行时），经过SQL优化、索引优化后仍无法满足性能要求；
  - 单库写并发压力大，连接数不够用
- **定义**
  - 分库：将数据按照一定规则分散到多个数据库实例中。不同库可以在不同服务器上
  - 分表：将一张大表按照一定规则拆分成多个结构相同的小表
- **作用**
  - 空间换时间：将集中式压力分散到多个节点，通过增加机器数量换取更高吞吐能力和更低延迟

分库

- **垂直分库**
  - 通常根据业务和功能的维度进行拆分，将不同业务数据分别存储在不同的数据库中，遵循核心理念“专库专用”
  - 作用：业务数据分流
- **水平分库**
  - 是将一个库的数据按照特定规则拆分到不同的数据库实例中（一般位于不同服务器上）。
  - 作用：减少单库数据量

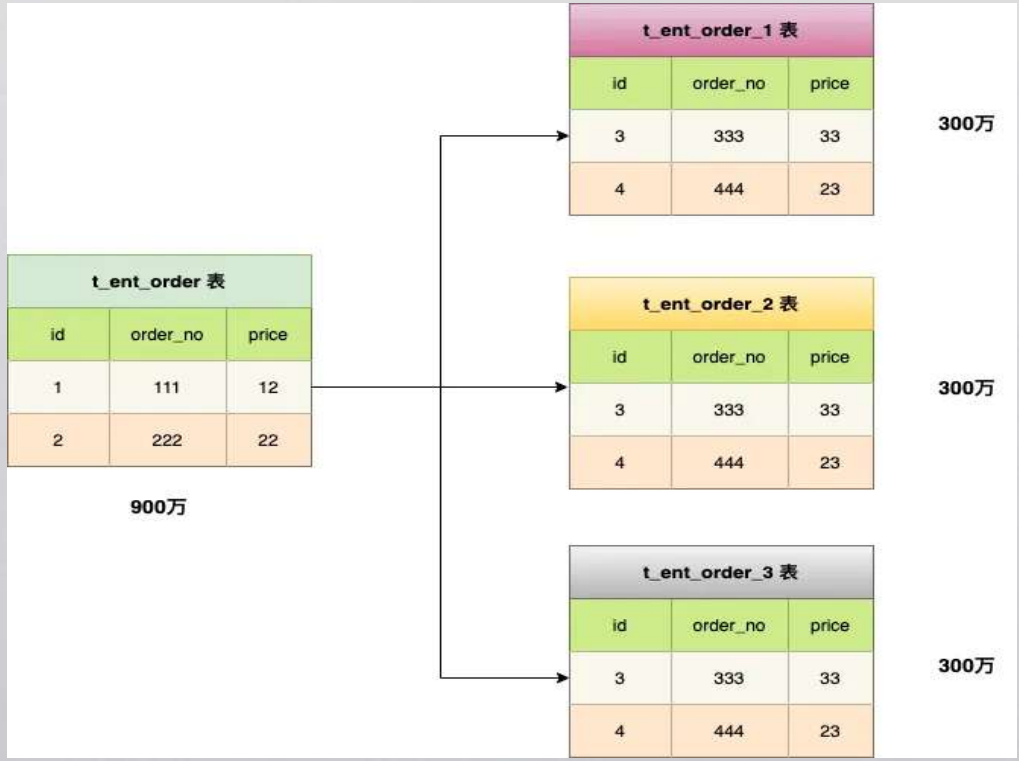




武漢大學 WUHAN UNIVERSITY

分表

- **垂直分表**
  - 针对业务上字段比较多的大表进行的，一般是把业务宽表中比较独立的字段，或者不常用的字段拆分到单独的数据表中，是一种大表拆小表的模式。
  - 作用：宽表变窄表
- **水平分表**
  - 在同一个数据库内，把一张大数据量的表按一定规则，切分成多个结构完全相同表，而每个表只存原表的一部分数据。
  - 作用：减少单表数据量



选择分库还是分表?

- **分库目的**
  - 提升数据库的存储容量、支撑更多的数据库连接，大幅度提升数据读写性能 （磁盘 IO 指标）
- **分表目的**
  - 减少单表的存储容量，提升数据的读写效率。分表并没有将拆分后的表分散到不同物理机器上，仍然在竞争同一个物理机的 CPU、内存、网络 IO 等资源，依然存在性能上限

场景	推荐方案
不同业务模块耦合在一起	垂直分库
单库连接数不够、磁盘空间不足	水平分库
表字段太多，且有些字段访问少	垂直分表
单表数据量过大	水平分表
综合场景	水平分库 + 水平分表（如订单库分4个库，每库4张表）

漢大學 WUHAN UNIVERSITY

分片（Sharding）策略

分片（Sharding）

- 水平分库和分表一般称为“数据分片”
- 基于“**分片键**”将数据均匀分散到多个数据库/表中

分片键选择三原则

- 离散性：分片键的值分布应尽可能均匀，避免数据倾斜（如订单状态字段不适合做分片键）
- 业务相关性：80%以上查询应能携带分片键，否则会导致全路由（需扫描全部分片，性能下降）
- 稳定性：分片键的值不应随业务变更而变化（如手机号可能换号；用户ID、订单ID更稳定）

分片策略

策略	数据均匀性	范围查询	扩容难度	实现复杂度	适用场景
哈希取模	☆☆☆☆	不支持	难	☆	用户信息、订单均匀分布，分片数固定
范围分片	☆☆ (可能倾斜)	支持	易	☆☆	时序数据、按业务分区
一致性哈希	☆☆☆☆	不支持	中	☆☆☆☆	动态扩缩容的缓存/数据库
基因分片	☆☆☆☆☆	不支持	中	☆☆☆	订单系统、需多维度查询

哈希取模——最经典、最均匀的分片策略

- **原理**
  - 对分片键（如用户ID）计算哈希值（Java中直接使用 hashCode）
  - 分片下标 = hash(分片键) % 分片总数
- **举例**
  - 例如：订单表中一共4个库，每个库8张表，选择userid作为分片键
  - 一个订单中userid为1006，则库号=1006 % 4 = 2， 库中表号=1006 % 8 = 6
- **优点**
  - 数据在库间和表间分布基本均匀
  - 只需一次取模计算，性能极高
  - 同一分片键的数据始终落在同一个物理表。如同一个用户的订单放在同一张表中

范围分片——按区间划分，支持范围查询

- **原理**
  - 分片 = 分片键值所在的预定义区间
  - 场景：时序数据（日志、监控、订单按时间归档），数据有明显地域或业务分区（如按省份分片）
- **优点**
  - 范围查询友好：可以快速定位到连续区间内的分片，适合按时间或ID范围查询
  - 扩容简单：新增分片只需调整区间边界，无需迁移历史数据
  - 实现直观：易于理解和配置
- **缺点**
  - 数据可能倾斜：如果业务分布不均（如新用户注册集中在某一时间段），可能导致某些分片数据量远大于其他分片

    热点问题：最新数据往往集中在最后一个分片，写入压力集中

一致性哈希——最小化扩容影响的分片策略

- 原理
  - 将分片键的哈希值映射到一个环（0 ~ 2^32-1）上，将每个物理分片（数据库节点）也映射到环上的多个点（虚拟节点），数据顺时针找到最近的节点，即为目标分片
  - 适用于动态扩缩容频繁的系统，如Redis Cluster
- 优点
  - 扩容影响小：增加或减少节点时，仅影响环上相邻节点的数据
  - 负载更均衡：通过虚拟节点，可使数据分布更均匀
- 缺点
  - 实现复杂度高：需要维护哈希环、虚拟节点映射
  - 范围查询不支持：仍需遍历全部分片
  - 节点移除时数据可能倾斜：节点下线，其数据转移到相邻节点，可能导致该节点负载突增

基因分片——支持多维度查询

- 原理
  - 在生成分布式ID时，将分片基因（如用户ID的部分位）嵌入ID中
  - 路由时直接从ID中提取基因，计算目标分片
  - 这样既可通过ID直接定位分片，也可通过用户ID（基因相同）定位同一分片
- 如何实现?
  - 以订单ID可设计二进制64位长整型：符号位1位，时间戳 41位，分片基因（如用户ID的后12位）12位，序列号 10位
  - 提取基因：gene = (orderId >> 10) & 0xFFF （提取中间12位）
  - 分片计算：dbIndex = gene % 4，tableIndex = gene % 8

基因分片——支持多维度查询

- **优点**
  - 支持多维度查询：通过订单号可直接定位，通过用户ID也可定位（因为用户ID与基因关联）
  - 数据均匀：基因来自用户ID，分布自然均匀
  - 无额外存储：无需维护分片映射表
- **缺点**
  - 需改造ID生成器：业务系统需统一使用基因ID生成器
  - 分片数上限：受基因位数限制（12位最多支持4096个分片）
- **适用场景**
  - 订单系统、消息系统等需要根据主键和业务键（如用户ID）查询的场景

基因分片——支持多维度查询

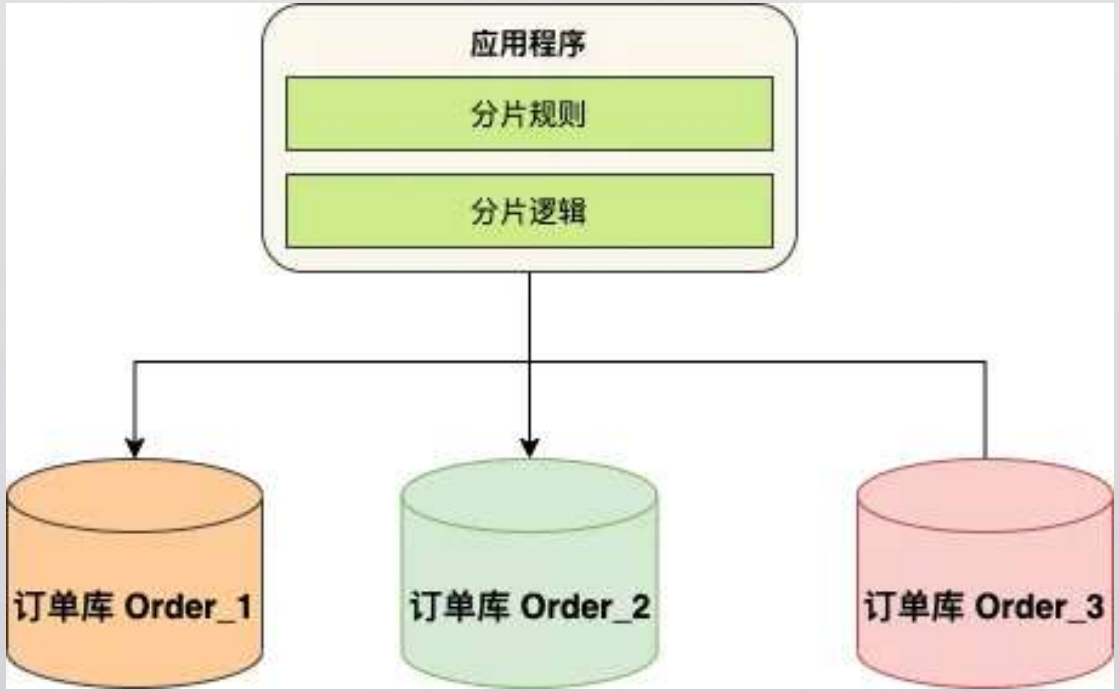
- (1L << 12) - 1 = 4095（二进制111111111111）：生成一个低12位全为1的掩码，用于提取低12位。
- userId & (1L << 12) - 1：保留用户ID的低12位，高位清零得到基因。

```java
public class OrderIdGenerator {
    // 基因位数：12位
    private static final int GENE_BITS = 12;
    public static long generateId(long userId) {
        // 1. 生成时间戳（相对某个纪元，例如Snowflake的纪元：1288834974657L）
        long timestamp = System.currentTimeMillis() - 1288834974657L;
        // 2. 提取用户ID的低12位作为基因
        //    (1 << GENE_BITS) - 1 生成低12位全为1的掩码，如 0xFFF
        long gene = userId & ((1L << GENE_BITS) - 1);

        // 3. 获取序列号（同一毫秒内自增，需处理并发）
        long sequence = getNextSequence(); // 伪代码，实际需结合时间戳实现

        // 4. 组合ID：
        //    时间戳左移（GENE_BITS + 10）= 22 位，为基因和序列号腾出空间
        //    基因左移 10 位，为序列号腾出空间
        //    序列号放在最低位
        return (timestamp << (GENE_BITS + 10)) | (gene << 10) | sequence;
    }

    // 从订单ID反推分片位置（提取基因）
    public static int getShardKey(long orderId) {
        // 右移10位（跳过序列号），然后取低12位（基因）
        return (int) ((orderId >> 10) & ((1L << GENE_BITS) - 1));
    }
}
```

分库分表的实现模式

（1）客户端模式

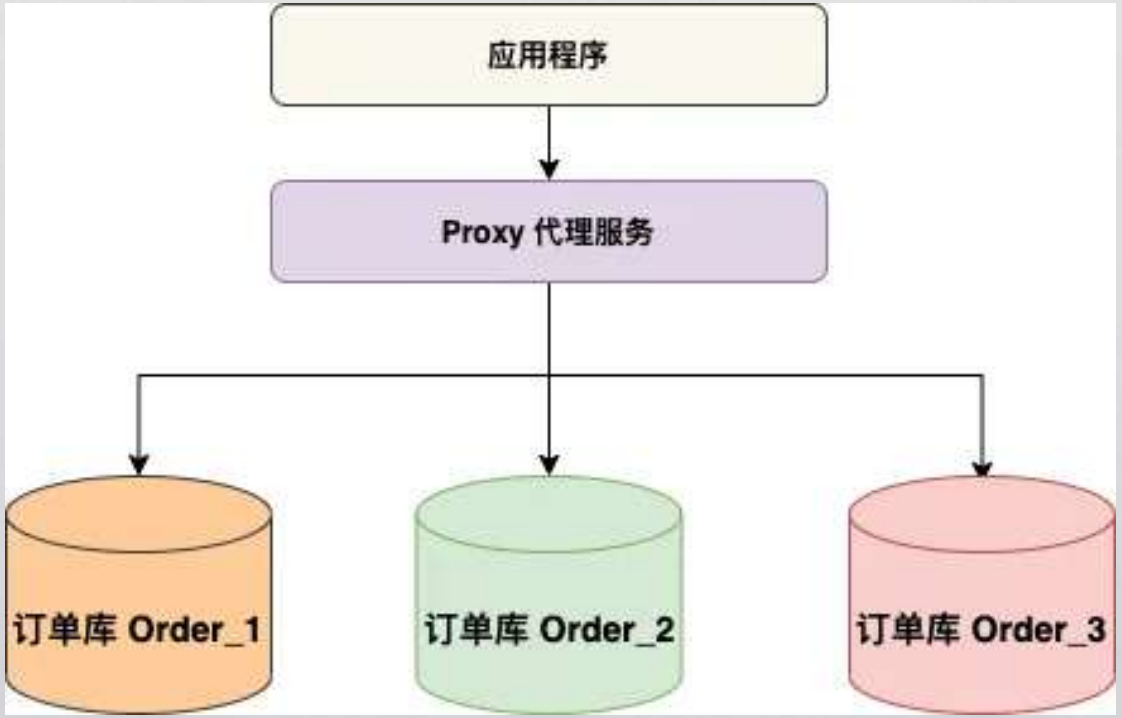
- 分库分表的逻辑完全由系统应用内部控制。应用直接连接多个数据库执行拆分后的 SQL 操作，然后在本地进行数据的合并和汇总等操作
- 代表产品：ShardingSphere-JDBC、TDDL
- 优点：直连数据库，额外开销小，实现简单
- 缺点：无法减少连接数消耗，有一定的侵入性，多数只支持 Java 语言



分库分表的实现模式

（2）代理模式

- 将应用程序与 数据库隔离，业务方的应用不需要直连数据库，而是连接代理服务，代理服务会将 SQL 分发到具体数据库执行并返回结果
- 代表产品：Mycat，ShardingSPhere Proxy
- 优点：应用零改动，和语言无关，可以通过连接共享减少连接数消耗。
- 缺点：因为有代理层，存在额外的时延



ShardingSphere-JDBC配置示例

ds0 和 ds1：分别配置了两个独立的 MySQL 数据库连接

```yaml
spring:
  shardingsphere:
    datasource:
      names: ds0,ds1                    # 两个数据源，对应两个数据库
      ds0:
        type: com.zaxxer.hikari.HikariDataSource
        driver-class-name: com.mysql.cj.jdbc.Driver
        jdbc-url: jdbc:mysql://loc
        username: root
        password: 123456
      ds1:
        type: com.zaxxer.hikari.Hi
        driver-class-name: com.mys
        jdbc-url: jdbc:mysql://loc
        username: root
        password: 123456
```

actual-data-nodes：定义了所有物理表的分布。

- ds$->{0..1}：表示数据源名称 ds0 和 ds1。
- t_order_$->{0..1}：表示在每个数据源中，存在 t_order_0 和 t_order_1 两张表。
- 语法中的 $->{...} 是 Groovy 表达式

```yaml
    rules:
      sharding:
        tables:
          t_order:                          # 逻辑表名
            actual-data-nodes: ds$->{0..1}.t_order_$->{0..1}  # 实际表分布
            database-strategy:
              standard:
                sharding-column: user_id      # 分库键
                sharding-algorithm-name: db-inline
            table-strategy:
              standard:
                sharding-column: order_id     # 分表键
                sharding-algorithm-name: table-inline
            key-generate-strategy:
              column: order_id                # 分布式主键生成列
              key-generator-name: snowflake   # 使用雪花算法生成ID
```

武漢大學 WUHAN UNIVERSITY

ShardingSphere-JDBC配置示例

```yaml
sharding-algorithms:
  db-inline:
    type: INLINE
    props:
      algorithm-expression: ds$->{user_id % 2}       # user_id % 2 决定数据源
  table-inline:
    type: INLINE
    props:
      algorithm-expression: t_order_$->{order_id % 2} # order_id % 2 决定表名
```

algorithm-expression:

- **ds$->{user_id % 2}：**根据 user_id 对 2 取模的结果，动态拼接出数据源名称。例如 user_id % 2 = 0 时，结果为 ds0；user_id % 2 = 1 时，结果为 ds1。
- **t_order_$->{order_id % 2}：**根据 order_id 对 2 取模的结果，动态拼接出表名，如 t_order_0 或 t_order_1。
- 注意：这里的分库键和分表键不同，根据用户id分库，根据订单号分表

分库分表的技术挑战

挑战类别	具体问题描述	可采取的解决方案
分布式事务	一次业务操作涉及多个数据库，无法使用单库事务保证ACID，可能出现数据不一致。	• XA协议（2PC）、TCC模式、Seata AT、本地消息表
跨库关联查询	需要JOIN的数据分布在不同的数据库或分片中，无法直接执行数据库JOIN。	• 字段冗余、全局表、应用层组装、搜索引擎
分页排序	跨分片执行LIMIT M,N时，需要从各分片拉取大量数据再合并排序，深分页性能极差。	• 禁止深分页、游标分页、使用搜索引擎
分布式ID生成	数据库自增ID在分库分表后无法保证全局唯一，且无法直接用于路由。	• 雪花算法、基因分片ID、Redis原子递增
数据倾斜	分片键选择不当导致某些分片数据量远大于其他分片，形成热点，影响整体性能。	• 重新选择分片键、复合分片键、虚拟节点、热点数据隔离
扩容与数据迁移	分片数不足以支撑未来数据量时需扩容，但取模分片下扩容会导致大量数据迁移。	• 预留分片、一致性哈希、双写迁移、停服迁移
跨库聚合统计	需要执行COUNT、SUM、AVG等聚合函数时，必须从全部分片拉取数据再计算。	• 应用层聚合、预计算、搜索引擎/OLAP库、业务降级

问题：跨库关联查询

问题描述

- 分库分表后，原本简单的 JOIN 操作变得困难。如果关联的表不在同一个库甚至不在同一个分片，无法直接使用数据库 JOIN

方案	实现方式	优点	缺点
字段冗余	在表中冗余存储关联表的字段	无需关联，查询快	数据冗余，更新成本高
全局表	将关联的小表（如字典表）在所有库中复制一份	可在本地 JOIN	只适用于数据量小、更新少的表
应用层组装	多次查询，应用层数据组装	灵活，适用于复杂场景	多次网络IO，性能较差
使用搜索引擎	将关联数据同步到 ES检索	支持复杂查询	引入额外组件，有延迟

实践建议

- 数据建模先行：在设计分片时，尽量将需要关联的数据放在同一分片（如按用户ID分片，用户订单和用户详情自然在一起）
- 业务维度拆分：按业务聚合，减少跨库关联
- 异构索引：使用 ES 等搜索引擎处理复杂关联查询

问题：分页与排序

问题描述

- 例如：SELECT * FROM t_order ORDER BY create_time LIMIT 100000, 10
- 在分库分表环境下，每个分片都要取 100010 条数据，然后在内存中合并排序，再截取 10 条
- 深分页是“性能杀手”：数据量越大，性能越差，甚至导致 OOM

方案	原理	适用场景
禁止深分页	限制只查询前几页	用户习惯场景（如电商列表）
游标分页（Search After）	记住上一页最后一条的排序字段值，下一页从该值开始	适合实时滚动加载
使用搜索引擎	将数据同步到 ES，由 ES 分页排序	复杂查询、大数据量
数据分片优化	在分片键中包含排序信息，减少需要合并的数据量	特定业务场景

问题：分页与排序

实践建议

- 业务设计优先：避免深分页，采用游标分页。
- 若必须深分页：引入 ES（内部支持 search_after 游标） 或 ClickHouse 等专门的分析引擎
- 分片键与排序字段一致：如果排序字段也是分片键（如 user_id），则可以按用户分片，但全局排序仍需要聚合。

游标分页（Search After）

```
-- 第一页
SELECT * FROM t_order ORDER BY create_time LIMIT 10;
-- 假设上一页最后一条记录的 create_time = '2023-01-01 12:00:00', id = 100
-- 下一页查询
SELECT * FROM t_order
WHERE create_time < '2023-01-01 12:00:00'
  OR (create_time = '2023-01-01 12:00:00' AND id > 100)
ORDER BY create_time DESC, id DESC
LIMIT 10;
```

武漢大學 WUHAN UNIVERSITY

问题：分布式ID生成

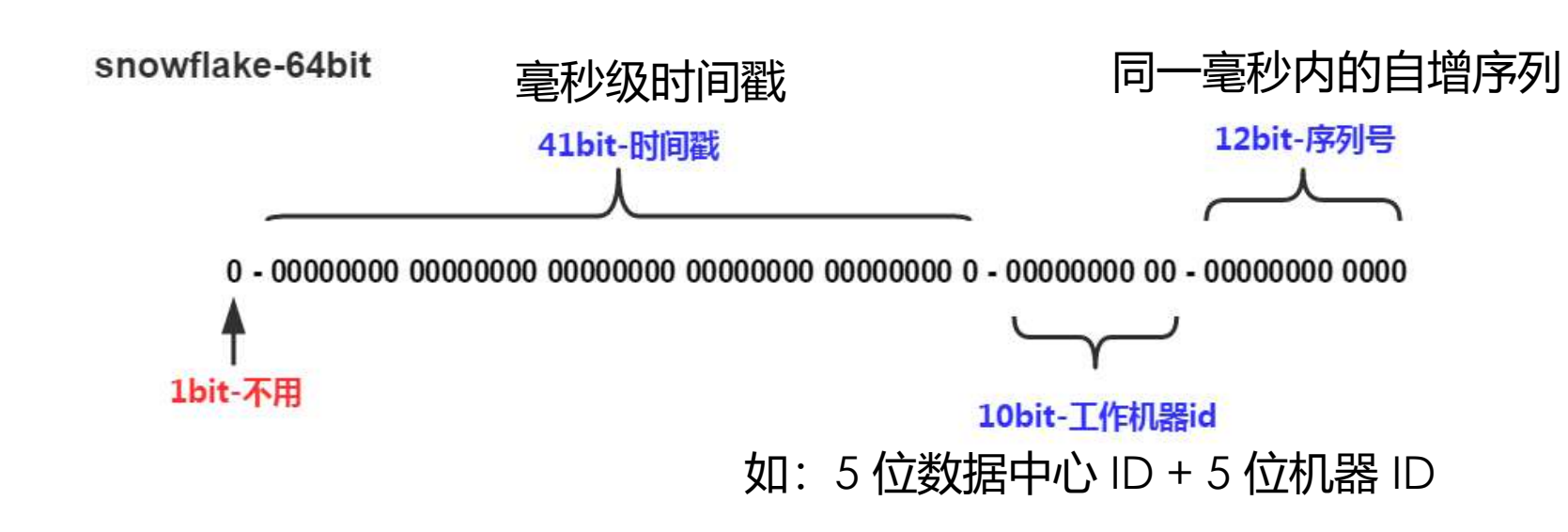
问题描述

- 分库分表后，数据库自增主键不再适用（可能重复）
- 需要一个全局唯一的ID生成器，同时最好能嵌入分片信息，便于路由

方案	ID特点	优点	缺点
UUID	36位字符串，无序	简单，无中心化	过长、无序，影响索引性能
数据库自增+步长	整数，趋势递增	简单，利用现有数据库	依赖数据库，扩展性差
Redis原子递增	整数，趋势递增	性能高，易实现	引入Redis组件，有网络开销
雪花算法	64位长整型，趋势递增	高性能，无中心化，可嵌入机器ID	依赖时钟，时钟回拨有问题
基因分片ID	64位，嵌入分片基因	可直接路由，支持多维度查询	需改造ID生成器

雪花算法

- 雪花算法
  - Twitter 开源的一种分布式 ID 生成算法，生成的 64 位 ID。



雪花算法

```java
/**
 * 生成下一个ID（线程安全）
 */
public synchronized long nextId() {
    long currentTimestamp = System.currentTimeMillis();
    // 处理时钟回拨
    if (currentTimestamp < lastTimestamp) {
        long offset = lastTimestamp - currentTimestamp;
        if (offset > 5) {
            throw new RuntimeException("时钟回拨超过5ms，拒绝生成ID"
        }
        // 若回拨时间较短，则等待时间追上
        try {
            wait(lastTimestamp - currentTimestamp);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new RuntimeException("等待时钟追赶时被中断");
        }
        currentTimestamp = System.currentTimeMillis();
    }
```

```java
    // 同一毫秒内，序列号自增
    if (currentTimestamp == lastTimestamp) {
        sequence = (sequence + 1) & MAX_SEQUENCE;
        if (sequence == 0) {
            // 当前毫秒序列号已用完，等待下一毫秒
            currentTimestamp = waitNextMillis(currentTimestamp);
        }
    } else {
        // 新的一毫秒，序列号重置为0
        sequence = 0L;
    }
    lastTimestamp = currentTimestamp;
    // 组合ID
    return ((currentTimestamp - START_TIMESTAMP) << TIMESTAMP_SHIFT)
            | (dataCenterId << DATA_CENTER_ID_SHIFT)
            | (workerId << WORKER_ID_SHIFT)
            | sequence;
}
```

问题：数据倾斜

问题描述

- 由于分片键选择不当或业务特点，导致部分分片数据量特别大，成为性能瓶颈
- 原因：分片键离散性差（如按地区分片，一线城市数据量大）、业务发展不均衡（如新用户集中在某时间段）、哈希取模时，某些分片键冲突较多。

方案	做法	效果
重新选择分片键	更换离散性更好的字段（如用户ID）	根治
复合分片键	使用多个字段组合分片	分布更均匀
虚拟节点（一致性哈希）	增加虚拟节点数，使分布更均匀	缓解但无法彻底解决
热点数据单独处理	将热点数据移出分片，使用缓存或单独表	临时方案
动态权重调整	监控数据量，动态调整分片策略	复杂，需自动迁移

问题：扩容与数据迁移

问题描述

- 随着业务增长，原有分片数不足以支撑数据量，需要扩容
- 但取模分片策略下，扩容会导致大部分数据需要迁移

方案	原理	迁移量	实施复杂度
停服迁移	暂停服务，迁移数据，恢复服务	100%	低，但影响大
双写迁移	同时写新旧库，逐步切换读流量	少量（增量）	高
一致性哈希扩容	采用一致性哈希，扩容只影响部分节点	1/N（N为原节点数）	中
虚拟节点+动态映射	提前分配大量虚拟节点，扩容时调整映射	极少	高（需中间件支持）

问题：跨库聚合与统计

问题描述

- 例如：统计所有订单的总金额（SUM）、用户数（COUNT）等
- 需要从全部分片中拉取数据，在应用层聚合

方案	做法	适用场景
应用层聚合	分别查询各分片，应用层汇总	简单统计，数据量不大
使用中间件	如 ShardingSphere 内置聚合功能	透明聚合，性能尚可
提前汇总（预计算）	定时任务汇总结果存入汇总表	大屏展示，对实时性要求不高
使用搜索引擎	同步到 ES 或 ClickHouse 进行 OLAP	复杂统计分析
业务降级	限制统计范围（如只查最近一个月）	业务可接受时

实践建议

- 对于实时性要求高的简单聚合，可使用中间件
- 对于复杂分析，建议引入ClickHouse 等列式数据库

数据库优化技术路线

- **核心原则**
  - 能不拆则不拆：先考虑优化索引、缓存、读写分离
  - 拆前充分评估：根据业务特点选择分片键和策略
  - 接受分布式的不完美：用最终一致性替代强一致性，用查询引擎解决复杂查询
- **优化次序**
  - 主从复制/读写分离 => 垂直拆分 => 水平分库分表 => 引入搜索引擎/NoSQL => 数据湖/数仓

作业：高并发写

消息队列

- 实现秒杀下单功能，建议Redis缓存库存，Kafka异步处理订单创建，削峰填谷。
- 使用雪花算法或基因算法生成订单ID，支持按用户ID或订单ID查询订单
- 幂等性：防止重复下单（同一用户同一商品只能秒杀一次）。
- 数据一致性：保证最终库存不超卖，订单数据完整。

分库分表（选做）

- 使用ShardingSphere-JDBC或ShardingSphere-Proxy，实现订单表分库分表，按用户ID分库，按订单ID分表